

Prática de Laboratório 2

Integração do Analisador Léxico ao Analisador Sintático Descendente Recursivo

Objetivos da Nova Prática



Processar Texto

Fazer o programa deixar de receber constantes numéricas diretamente e passar a aceitar expressões em formato de texto bruto.



Construir o Lexer

Implementar a varredura da string (Scanner) para identificar padrões e gerar os *tokens* correspondentes em tempo de execução.



Integrar as Fases

Conectar o analisador léxico recém-criado com a estrutura do analisador sintático construído na primeira prática.

Contextualização: O Problema Atual

- Na **Prática 1**, simulamos a entrada injetando uma sequência de identificadores manualmente no código (ex:).

```
IDENT, MAIS, NUMERO ...
```

- Em um compilador real, o usuário não fornece tokens, ele fornece o **código-fonte** em formato de string.

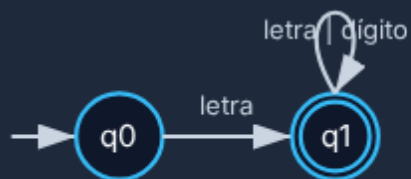
- O nosso programa precisa agora ser capaz de receber via teclado ou arquivo.

```
"A + B * 32"
```

- É necessário um módulo intermediário para ler os caracteres, ignorar espaços e classificar cada fragmento válido.

Descrição de Símbolos com Autômatos Finitos

Identificadores



Números



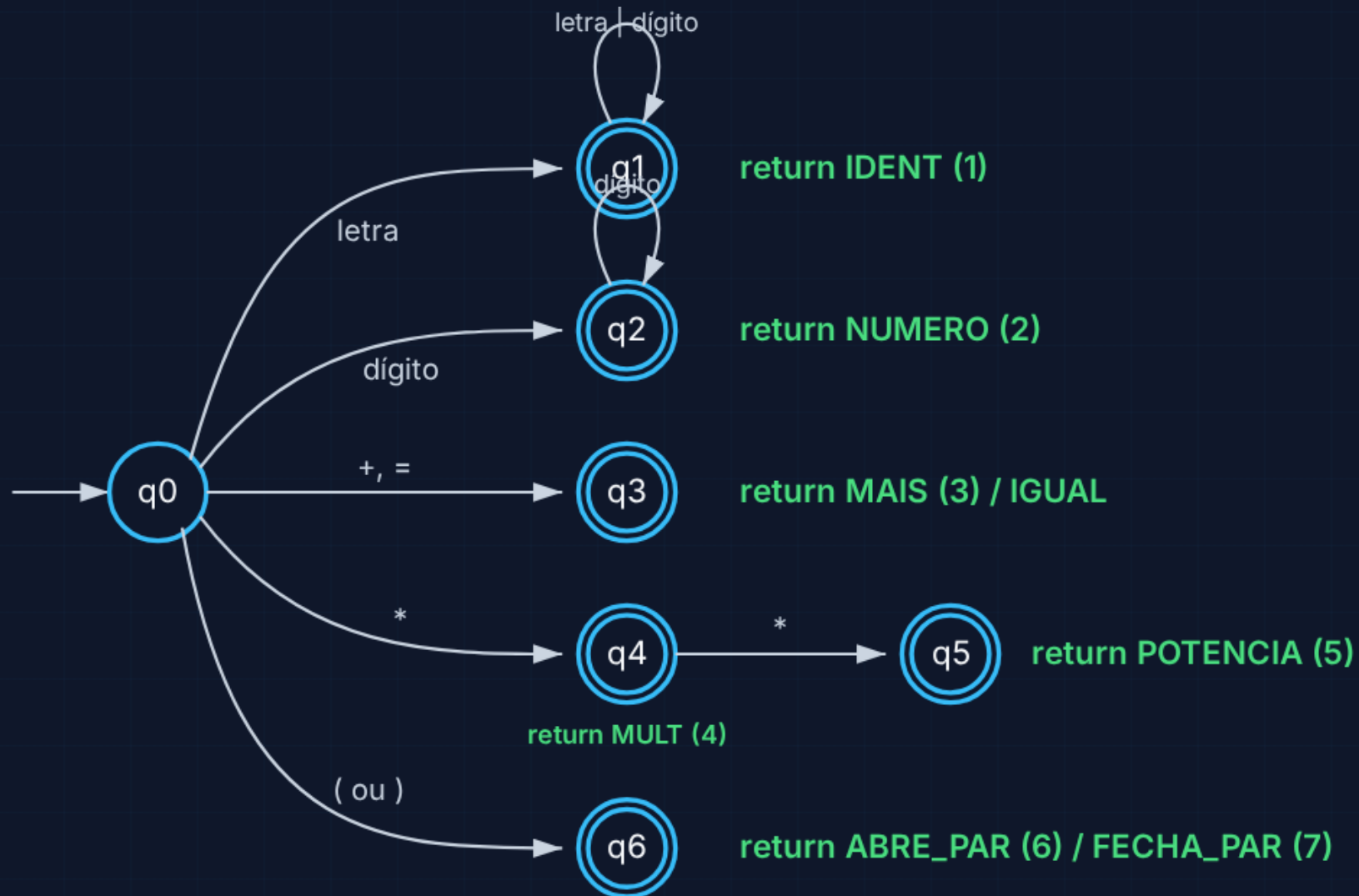
Operadores



Parentizadores



O Autômato **Unificado** do Analisador Léxico



A Fase Léxica (Scanner)

Transformando fluxo de caracteres em fluxo de tokens

Regras para os Tokens



Identificadores (IDENT): Sequências que começam obrigatoriamente com uma letra e podem ser seguidas por letras ou números.

Ex:

A Var1 somaTotal



Números (NUMERO): Sequências compostas exclusivamente por dígitos numéricos.

Ex:

42 0 1024



Operadores e Pontuação: Símbolos literais definidos na gramática.

Ex: + (MAIS), * (MULT), ** (POTENCIA), ((ABRE_PAR),) (FECHA_PAR).



Espaços em Branco: O analisador léxico deve consumir e ignorar espaços, tabulações e quebras de linha antes de reconhecer o próximo token.

O Modelo de Comunicação

Analizador Léxico (Scanner)

Atua sob demanda. Mantém um ponteiro global para a posição atual da string de entrada. Quando solicitado, avança lendo caracteres, agrupa um lexema válido, retorna o código numérico do *token* correspondente e pausa a leitura.

Analizador Sintático (Parser)

O mestre do processo. Quando a função for chamada, ela `obtenha_simbolo()` um vetor estático. Ela fará uma chamada ao Léxico pedindo: "Me dê o próximo token válido disponível nesta string".

Fluxo de Execução

O fluxo tradicional da compilação é estabelecido através de chamadas de funções aninhadas.

- 1 O programa principal lê a string de entrada via terminal.
- 2 Chama-se `obtenha_simbolo()` para ler o primeiro token.
- 3 Inicia-se a árvore chamando. `EXPR()`
- 4 Durante as regras recursivas, novos tokens são solicitados via léxico.
- 5 Conclui-se com Sucesso ou Erro Sintático/Léxico.

A Tarefa de Implementação



Entrada de Dados

Atualize a `main()` para ler uma linha de texto do teclado (ex: `fgets`) e armazenar em um array de ``char`` global.



Função Scanner

Crie uma função `proximo_token()` que implemente a lógica dos autômatos para varrer a string global caractere a caractere.



Atualizar `obtem_simbolo`

Modifique a antiga rotina para que ela apenas chame e atribua à variável global.

```
obtenha_simbolo() proximo_token()
```

Casos de Teste (Integração)

Expressão de Entrada (String)	Tokens Gerados (Lexer)	Resultado Sintático
"varX + 45"	IDENT, MAIS, NUMERO, FIM	Expressão Válida
"A * (B + 2)"	IDENT, MULT, ABRE, IDENT, MAIS, NUMERO, FECHA, FIM	Expressão Válida
"soma + "	IDENT, MAIS, FIM	Erro Sintático (esperava termo)
"var1 \$ 3"	IDENT, ERRO LÉXICO	Erro Léxico (caractere '\$')

Perguntas de Reflexão

- ? O que acontece na sua implementação se o usuário digitar espaços extras no meio da expressão, como: `A + B`? Onde esse tratame `A + B` rrer?
- ? Como o Lexer diferencia o operador (MULT) `*` operador (POTENCIA) sei `**` usar ambiguidade? O que é o `*lookahead*` (olhar adiante)?
- ? Se uma variável contiver um caractere inválido (ex: `v@lor`), qual fase (léxica ou sintática) será responsável por interromper a execução e gerar a mensagem de erro? Justifique.

Entrega

O que deve ser entregue:

- Código-fonte final contendo o analisador léxico integrado ao sintático.
- Prints da execução rodando os 4 casos de teste de integração mostrados anteriormente.
- Um PDF curto contendo as respostas para as Perguntas de Reflexão.

Bom trabalho!